

Flush+Monitor su Linux moderno

Riproduzione del paper Eviction Notice, adattamento a Linux 7.2
e valutazione di una mitigazione constant-access

5.15

referimento del Monitor originale

7.2

kernel moderno validato

825 / 850 μ s

finestre minime confermate

Perché esiste la page cache?

DISCO / SSD

Persistente, capiente, ma molto più lento della RAM.



PAGE CACHE · RAM

inode X ·
pagina 0

pagina 1

pagina 2



PROCESSI

read(), pread(), mmap() riusano la copia già residente.

4 KiB

risoluzione tipica di una pagina

stesso inode

stesso stato condiviso

hit / miss

metadato osservabile

La cache è condivisa per efficienza: librerie e binari comuni non vengono duplicati per ogni processo.
Proprio questa condivisione crea il canale laterale.

Da ottimizzazione a canale laterale

ATTACCANTE

1 · Flush

Rimuove la pagina target dalla page cache.

3 · Monitor

Osserva se la pagina è tornata residente.

FINESTRA DI OSSERVAZIONE

EVENT = 0

La victim non legge.
La pagina resta assente.

EVENT = 1

La victim legge.
La pagina torna in RAM.

INFERENZA

miss → EVENT 0

hit → EVENT 1

Nessuna lettura della memoria privata della victim: si osserva soltanto la residency della pagina condivisa.

Threat model: attaccante locale non privilegiato, utente differente, stessa macchina, accesso in lettura allo stesso file o alla stessa libreria condivisa.

Il vocabolario del paper: quattro primitive

FLUSH

Rimozione deterministica e non privilegiata
POSIX_FADV_DONTNEED

EVICT

Rimozione indiretta tramite pressione di memoria

MONITOR

Residency senza caricare una pagina assente

RELOAD

Tempo di lettura: lento dal disco, veloce dalla RAM

Le primitive si combinano in cinque tecniche generiche

Flush+Monitor

Flush+Reload

Flush+Flush

Evict+Monitor

Evict+Reload

Flush+Monitor è il caso centrale: rimozione affidabile + osservazione diretta, senza dipendere dal timing di una lettura.

Mitigazioni: interfacce chiuse, canale ancora aperto



**Le patch ad hoc riducono una via di osservazione, ma la page cache continua a essere condivisa.
Il problema si sposta da una syscall alla successiva.**

Flush+Monitor nel paper Eviction Notice

1

FLUSH

pagina fuori cache

2

ATTESA

la victim può accedere

3

MONITOR

hit oppure miss

SEMANTICA ATTESA

hit → preadv2 restituisce 1 byte

miss → -EAGAIN e la pagina resta assente

0,8 μ s

limite della primitiva sul sistema del paper

4 KiB

risoluzione spaziale

37,7 kB/s

covert channel con cachestat

Il paper valuta Linux 6.8 e verifica anche 6.12.4.

I risultati più veloci di Flush+Monitor usano cachestat; preadv2 è l'alternativa cross-user.

La classificazione è ancora corretta. Il post-stato del probe non lo è più.

Domanda sperimentale: possiamo mantenere Flush+Monitor utilizzabile su Linux 7.2 senza scambiare la contaminazione del probe per attività della victim?

Risposta: sì, classificando il primo ritorno e ripristinando la cache prima del round successivo.

Su Linux 7.2, un miss può contaminare il round successivo

1 · PROBE

`preadv2(..., RWF_NOWAIT)`



2 · MISS

ritorno immediato
-EAGAIN



3 · READAHEAD

I/O asincrono avviato dal
kernel



4 · CACHE = 1

la pagina compare dopo il
ritorno

Commit 0a2d8294 · “mm: allow read-ahead with IOCB_NOWAIT set”

Rimuove l'associazione automatica `RWF_NOWAIT` → `IOCB_NOIO` e permette di avviare readahead senza attendere l'I/O.

5.15

miss → pagina ancora assente

7.2

miss → fill asincrono osservato

Disegno sperimentale: una pagina, tre ruoli

ATTACKER · vCPU 0

Flush → attesa → Monitor
Registra solo la predizione.

VICTIM · vCPU 1

EVENT 0: nessuna lettura
EVENT 1: pread della pagina.

VALIDATOR

Controlla cache iniziale, cleanup
e ordine temporale.

target.bin · 1 pagina · 4096 byte · stesso inode

ground truth separato dai dati dell'attaccante

5.15.0-187

VM reference · Ubuntu Jammy/ext4

7.2.0-flushmon

VM modern · Ubuntu Jammy/ext4

10.000

prove per finestra nello sweep

Due Monitor moderni che ripristinano lo stato

A · WAIT + FLUSH

CICLO

1. Il primo ritorno classifica hit o miss.
 2. Sul miss aspettiamo che il fill asincrono termini.
 3. POSIX_FADV_DONTNEED rimuove la pagina.
- ✓ Funziona su ext4 e nella demo Btrfs.

B · RWF_DONTCACHE

CICLO

1. Probe con RWF_NOWAIT | RWF_DONTCACHE.
 2. Il kernel richiede il drop-behind del contenuto letto.
 3. Il cleanup rende stabile il round successivo.
- ✓ Leggermente più rapido su ext4.
✗ Btrfs restituisce EOPNOTSUPP.

La predizione usa sempre il primo ritorno; il lavoro successivo serve soltanto a preparare il prossimo round.

Riproduzione riuscita e ciclo moderno misurato

200 / 200

upstream · Linux 5.15

200 / 200

wait+Flush · Linux 7.2

200 / 200

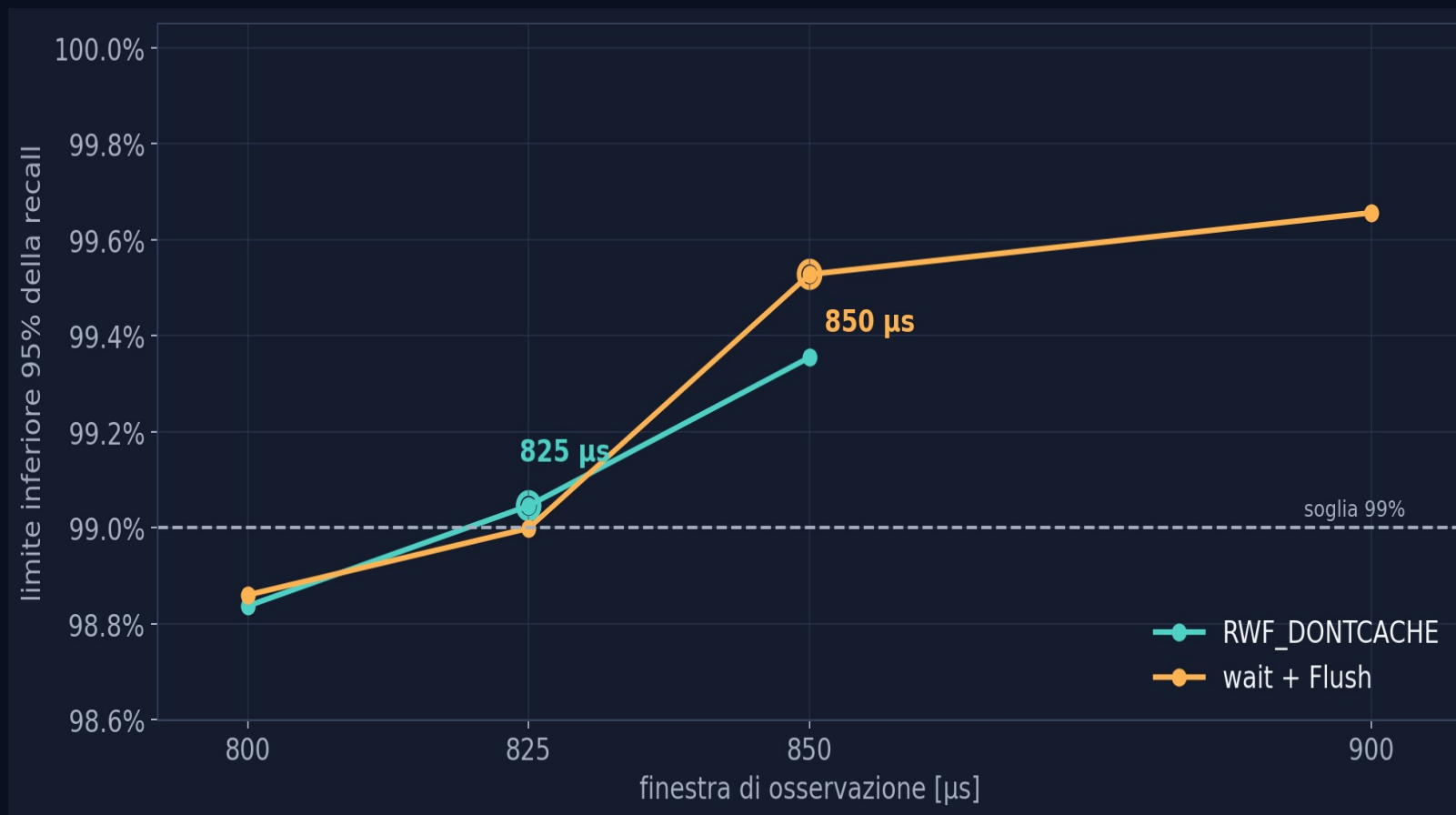
DONTCACHE · Linux 7.2

Tempo del ciclo isolato · mediana

Monitor	miss	hit
reference 5.15	0,232 µs	0,777 µs
modern wait+Flush	13,689 µs	0,814 µs
modern DONTCACHE	13,169 µs	0,820 µs

Il costo aggiuntivo è concentrato sul miss: dobbiamo attendere e ripulire il readahead.

Sweep: la finestra minima affidabile



825 μs

RWF_DONTCACHE · minimo confermato

850 μs

wait+Flush · minimo confermato

CRITERIO

Recall e accuracy $\geq 99\%$

FPR $\leq 1\%$

10.000 prove

limiti Wilson al 95%

La finestra immediatamente inferiore è stata provata e respinta: non stiamo scegliendo semplicemente il valore migliore osservato.

Paper e nostro lavoro: due metriche diverse

PAPER NDSS 2026

PRIMITIVA / CAPACITÀ

- Flush+Monitor principalmente con cachestat
- upper bound della primitiva: 0,8 μ s
- 37,7 kB/s nel covert channel
- Linux 6.8; verifica anche su 6.12.4
- obiettivo: massima risoluzione e attacchi applicativi

QUESTO PROGETTO

FINESTRA END-TO-END

- Monitor preadv2 con ripristino dello stato
- 825 / 850 μ s affidabili su 10.000 prove
- attacker + victim + scheduling + cleanup
- Linux 7.2 su ext4
- obiettivo: compatibilità moderna e ripetibilità dei round

0,8 μ s e 825/850 μ s non sono un confronto diretto di velocità.

Mitigazione: rendere l'accesso indipendente dall'evento

VICTIM NORMALE

EVENT = 0

nessuna read
→ miss

EVENT = 1

pread
→ hit

VICTIM CONSTANT-ACCESS

EVENT = 0

dummy pread
→ hit

EVENT = 1

pread reale
→ hit

La residency codifica 1 bit.

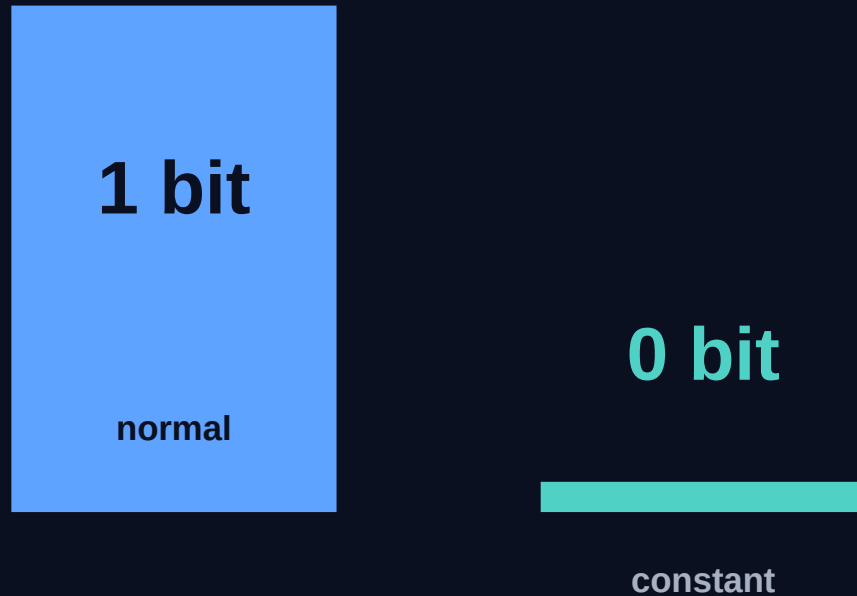
La residency è costante.

La mitigazione non impedisce alla pagina di entrare in cache.

Fa sì che entri in cache in entrambi i casi, eliminando la dipendenza dall'informazione sensibile.

Sicurezza ottenuta e costo misurato

Informazione osservabile



Overhead assoluto della victim

+380,73 ns

EVENT 0 · dummy read

-9,17 ns

EVENT 1 · rumore

+185,78 ns / op

media bilanciata

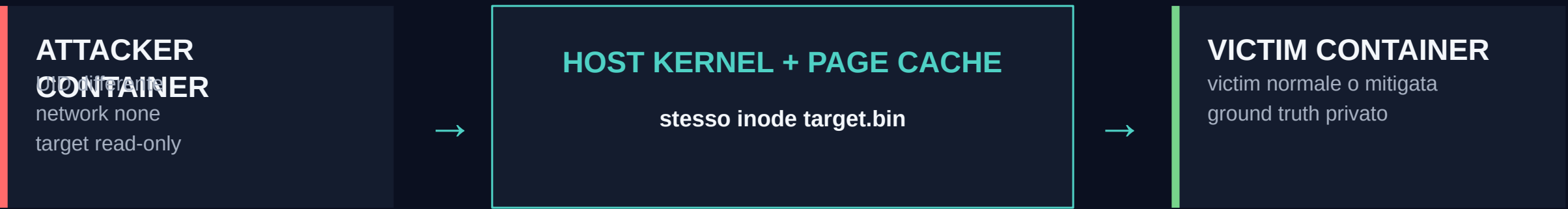
COME LEGGERLO

100.000 operazioni per policy, pagina già calda.

Conta il costo assoluto: la percentuale di EVENT 0 è grande perché il baseline dura circa 50 ns.

Accuracy 50% da sola non basta: il risultato di sicurezza è prediction costante + mutual information pari a zero.

Demo: lo stesso canale attraversa i container



policy	event	osservazione
normal	0	miss
normal	1	hit
constant	0	hit
constant	1	hit

```
./demo/cross-container.sh
```

Demo opzionale UI redressing: rilevamento della finestra Zenity e overlay innocuo.

Conclusioni

01

La page cache è metadato condiviso

Processi e container separati osservano lo stesso stato di residency.

02

Le mitigazioni ad hoc spostano l'attacco

mincore e cachestat sono state limitate; altre primitive restano disponibili.

03

preadv2 richiede oggi un restore cycle

Su Linux 7.2 il miss può avviare readahead; wait+Flush e DONTCACHE ristabiliscono lo stato.

04

La mitigazione elimina il bit

Constant-access porta la mutual information da 1 a 0 bit con +185,78 ns medi.

CONTRIBUTO DEL PROGETTO

Riproduzione del paper su 5.15, analisi del cambiamento moderno, due Monitor state-restoring su 7.2, sweep della finestra e mitigazione misurata.

LIMITI

Esperimento controllato su una pagina; valori dipendenti da sistema e filesystem; RWF_DONTCACHE non è universale; constant-access non è una difesa generale.

Fonti principali

- Eviction Notice · NDSS 2026 · doi.org/10.14722/ndss.2026.240006
- CVE-2025-21691 · cachestat permission checking
- Linux commit 0a2d8294 · IOCB_NOWAIT readahead
- Linux preadv2(2) · RWF_DONTCACHE
- Dati e demo: repository flush-monitor

Domande?